601.433/633 Introduction to Algorithms **Fall 2026**
Homework #7 **Due:** November 5, 2026, 11:59pm

Remember: this is an oral presentation homework! You are required to work in groups of three, and may collaborate only with other students in your group. We encourage you to write up solutions, but your grade will be based entirely on your presentation. At your presentation, the grader will randomly assign each student to present a problem. You should be able to create a group and sign up for a slot using Canvas. The deadline for signing up for a presentation slot is 5pm Wed Nov 4.

You may use the internet or LLMs to look up formulas, definitions, concepts, etc., but may not simply look up the answers online or ask an LLM.

1 Faster Shortest Paths

Let $G = (V, E)$ be a directed graph with lengths $\ell : E \to \mathbb{R}$ with no negative-length cycles. Let $v \in V$, and let α denote the maximum, over all $u \in V$, of the number of edges on a shortest path from v to u (where the shortest-path is defined with respect to the weights). Given G, w, and v (but not α), give an algorithm that computes shortest path distances from v to all other nodes in $O(m\alpha)$ time. You may assume that $m = \Omega(n)$. Prove correctness and running time.

Algorithm Description (Bellman–Ford with early termination):

Write $\delta(u)$ for the shortest-path distance from v to u ($\delta(u) = \infty$ if u is unreachable). Let α be the maximum over reachable u of the minimum number of edges on a shortest v–u path; so every reachable u has a shortest path with at most α edges. We assume $n \geq 2$ and at least one other vertex is reachable, so $\alpha \geq 1$ (otherwise the algorithm below costs $O(m)$ trivially).

(a) Set $d[v] = 0$ and $d[u] = \infty$ for all $u \neq v$.

(b) Repeat the following round: for every edge $(x, y) \in E$, if $d[x] + \ell(x, y) < d[y]$, set $d[y] \leftarrow d[x] + \ell(x, y)$. Track whether any d-value changed during the round. (Relaxation)

(c) If a round makes no change, stop and output d.

(We never need to know α in advance; the stopping rule detects it.)

Correctness Proof:

Claim 1 (Upper bound is never violated) *At all times, $d[u] \geq \delta(u)$ for all u.*

Proof: Induct on the number of relaxation steps performed so far. Base case (no relaxations yet): $d[v] = 0$ and $\delta(v) = 0$, so $d[v] \geq \delta(v)$. Every other $d[u] = \infty \geq \delta(u)$. Inductive step: Assume $d[u] \geq \delta(u)$ for all u before some relaxation step on an edge (x, y). Only $d[y]$ can change, so it suffices to check y. If $d[y]$ does not change, the inequality still holds by the hypothesis. If $d[y]$ changes, the new value is $d[x] + \ell(x, y)$, and it is finite, so $d[x]$ is finite. Then

$$d[y]_{\text{new}} = d[x] + \ell(x, y) \geq \delta(x) + \ell(x, y) \geq \delta(y)$$

The first inequality is the inductive hypothesis applied to x. The second is the triangle inequality $\delta(y) \le \delta(x) + \ell(x, y)$: a shortest path to x followed by the edge (x, y) is a walk to y, and since G has no negative cycles, any walk from v to y has length at least $\delta(y)$ (deleting cycles from the walk never increases its length and leaves a simple path). So the inequality holds after the step, completing the induction. ■

Claim 2 *After round i, for every u having a shortest path from v with at most i edges, $d[u] = \delta(u)$.*

Proof: Induction on i. For $i = 0$: only $u = v$, and $d[v] = 0 = \delta(v)$. Suppose it holds after round $i - 1$, and let u have a shortest path $v = x_0, x_1, \ldots, x_k = u$ with $k \le i$ edges. Then $x_0, \ldots, x_{k-1}$ is a shortest path to x_{k-1} with $k - 1 \le i - 1$ edges (subpaths of shortest paths are shortest), so $d[x_{k-1}] = \delta(x_{k-1})$ after round $i - 1$. In round i the edge (x_{k-1}, u) is relaxed, giving $d[u] \le d[x_{k-1}] + \ell(x_{k-1}, u) \le \delta(x_{k-1}) + \ell(x_{k-1}, u) = \delta(u)$. Combined with Claim 1, $d[u] = \delta(u)$. ■

By Claim 2, after round α we have $d[u] = \delta(u)$ for every reachable u; unreachable vertices keep $d[u] = \infty = \delta(u)$ throughout (an edge from a vertex with $d = \infty$ never triggers a relaxation, and unreachable vertices have no reachable in-neighbors).

Termination: In round $\alpha + 1$, every edge (x, y) satisfies $d[x] + \ell(x, y) \ge d[y]$ because $d = \delta$ and δ obeys the triangle inequality. So no change occurs and the algorithm stops after at most $\alpha + 1$ rounds.

Output. The algorithm only stops after a round with no change, but by the above it must stop by round $\alpha + 1$, at which point $d = \delta$. Hence the output is correct.

Running Time Proof:

Initializing $d[u]$ for all $u \in V$ takes $O(n) = O(m)$. Each round scans all m edges in $O(m)$ time. There are at most $\alpha + 1$ rounds, for a total of

$$O(n) + O(m(\alpha + 1)) = O(m\alpha),$$

using $m = \Omega(n)$ and $\alpha \ge 1$.

2 Multi-Set Shortest Paths

Let $G = (V, E)$ be a directed graph with weighted edges; edge weights can be positive, negative, or zero. There are no negative-weight cycles. Suppose the vertices of G are partitioned into k disjoint subsets $V_1, V_2, \ldots, V_k$; that is, every vertex of G belongs to exactly one subset V_i. For each $i, j \in [k]$, let $\delta(i, j)$ denote the minimum shortest-path distance between vertices in V_i and vertices in V_j:

$$\delta(i, j) = \min \{d(v_i, v_j) \mid v_i \in V_i \text{ and } v_j \in V_j\}$$

Design an algorithm to compute $\delta(i, j)$ for all $i, j \in [k]$ that runs in time $O(mn + kn \log n)$. You may assume that $m = \Omega(n)$. Prove correctness and running time.

Answer: Let $d(a, b)$ denote shortest-path distance in G (with $d(a, b) = \infty$ if b is unreachable from a).

Algorithm

(a) Add a new vertex s' with a 0-weight edge (s', x) for every $x \in V$. Run Bellman–Ford from s' to obtain $h(x) = d(s', x)$ for all $x \in V$. (No negative cycle is created, since s' has no incoming edges.)

(b) For each edge $(a, b) \in E$ define $w'(a, b) = w(a, b) + h(a) - h(b)$. (Reweighting)

(c) For each $i \in [k]$: build the graph G_i consisting of G with weights w' plus a new vertex s_i and edges (s_i, u) of weight $-h(u)$ for every $u \in V_i$. Run Dijkstra from s_i to get $D_i(x)$ for all $x \in V$.

(d) For each $i, j \in [k]$ output

$$\delta(i, j) = \min_{x \in V_j} \big(D_i(x) + h(x)\big).$$

Correctness

Step 1. Since G has no negative cycles and s' has no incoming edges, $G \cup \{s'\}$ has no negative cycles, so Bellman–Ford correctly computes h. All $h(x)$ are finite, and $h(x) \leq 0$ because of the edge (s', x) of weight 0.

Step 2. For each edge (a, b), $h(b) \leq h(a) + w(a, b)$ (triangle inequality), so $w'(a, b) = w(a, b) + h(a) - h(b) \geq 0$. Moreover, For any path $P : a = x_0, \ldots, x_r = b$ in G,

$$w'(P) = \sum_{t=1}^{r} \big(w(x_{t-1}, x_t) + h(x_{t-1}) - h(x_t)\big) = w(P) + h(a) - h(b).$$

Hence the shortest a–b paths under w and w' are the same, and $d'(a, b) = d(a, b) + h(a) - h(b)$.

Step 3. The weights $-h(u) \geq 0$ are nonnegative, so G_i has all nonnegative weights and Dijkstra is correct. The vertex s_i has no incoming edges, so it creates no new paths between vertices of V. Every $s_i \to x$ path has the form $s_i \to u \to x$ for some $u \in V_i$, with length

$$-h(u) + d'(u, x) = -h(u) + d(u, x) + h(u) - h(x) = d(u, x) - h(x).$$

Minimizing over $u \in V_i$ (and over paths from u to x),

$$D_i(x) = \min_{u \in V_i} d(u, x) - h(x).$$

The $h(u)$ terms cancel, which is exactly why the source edges carry weight $-h(u)$.

Step 4. Therefore

$$\min_{x \in V_j} \big(D_i(x) + h(x)\big) = \min_{x \in V_j} \min_{u \in V_i} d(u, x) = \delta(i, j).$$

(If no vertex of V_j is reachable from V_i, both sides are ∞. For $i = j$ we get $\delta(i, i) = 0$, as $d(u, u) = 0$.)

Running time

- Bellman–Ford on $n + 1$ vertices and $m + n$ edges: $O((n + 1)(m + n)) = O(mn)$ (using $m = \Omega(n)$).
- Reweighting: $O(m)$.

- Each Dijkstra on G_i has $n+1$ vertices and $m+|V_i| \le m+n = O(m)$ edges, so it costs $O(m+n\log n)$. Over k sets: $O(km + kn\log n)$. Since $k \le n$, $km \le mn$, so this is $O(mn + kn\log n)$.
- Recovering $\delta(i,j)$: for each i, compute $D_i(x) + h(x)$ for all x and take a minimum within each V_j. This is one pass of $O(n)$ per i, giving $O(kn)$ in total.

Total: $O(mn + kn\log n)$.

3 Single-Source Shortest Paths

Let $G = (V, E)$ be a connected, undirected graph with positive edge lengths $\ell : E \to \mathbb{R}^+$. This graph represents a road network. You are trying to get from $s \in V$ to $t \in V$. Unfortunately, you are a coffee addict: getting caffeine often enough is the most important thing to you. In fact, there is some number $L \in \mathbb{R}^+$ so that if you go more that L distance without getting coffee, you will get a caffeine migraine and crash your car. So you simply *cannot* go more than L distance without stopping for coffee. Fortunately, there is a coffee shop at every vertex $v \in V$ (including s and t) where you can get coffee. However, you are also a coffee snob, and you *hate* Starbucks. And there is a special subset of nodes $S \subseteq V$ where the only coffee shop on that node is a Starbucks.

So you have the following problem: given a subset $S \subseteq V$ of vertices where the only coffee shop is a Starbucks, find the path from s to t which uses only edges of length at most L, and has the *fewest* vertices in S. You may assume that $s, t \notin S$. Design an $O(m + n\log n)$-time algorithm for this problem, and prove running time and correctness.

Algorithm Description:

(a) Let $G' = (V, E')$ where $E' = \{e \in E : \ell(e) \le L\}$.

(b) Replace each undirected edge $\{u, v\} \in E'$ by two directed edges (u, v) and (v, u). Define the weight of an arc by

$$w(u,v) = \begin{cases} 1 & \text{if } v \in S, \\ 0 & \text{if } v \notin S. \end{cases}$$

(c) Run Dijkstra's algorithm from s on the directed graph with weights w.

(d) If t is unreachable from s (i.e. $d(t) = \infty$), report that no valid path exists. Otherwise, follow the parent pointers from t back to s and output the resulting path.

Correctness

Claim 3 *A path P from s to t in G uses only edges of length at most L if and only if P is a path from s to t in G'.*

Proof: By definition, E' consists exactly of the edges of G of length at most L, and G' has the same vertex set as G. Hence a path in G uses only such edges if and only if every one of its edges lies in E', i.e. if and only if it is a path in G'. ∎

Claim 4 *Let $P = (s = v_0, v_1, \ldots, v_k = t)$ be a path in G'. Then the total w-weight of P equals the number of vertices of P that lie in S.*

Proof: The weight of P is $\sum_{i=1}^{k} w(v_{i-1}, v_i) = \sum_{i=1}^{k} [v_i \in S]$, which counts the vertices among $v_1, \ldots, v_k$ that lie in S. Since $s = v_0 \notin S$, the vertex v_0 contributes nothing to the count of S-vertices on P, so this sum equals $|V(P) \cap S|$. ■

Theorem 5 *The algorithm outputs a path from s to t that uses only edges of length at most L and has the fewest vertices in S among all such paths. If no such path exists, the algorithm reports this.*

Proof: By Claim 3, the feasible paths (those using only edges of length at most L) are exactly the s–t paths in G'. If there are none, then t is unreachable from s in G', so Dijkstra's algorithm gives $d(t) = \infty$ and the algorithm correctly reports that no valid path exists.

Otherwise, all arc weights w are nonnegative, so Dijkstra's algorithm correctly computes a minimum-w- weight path P^* from s to t in G'. By Claim 4, the w-weight of any s–t path equals the number of its vertices in S. Hence P^* minimizes the number of S-vertices over all s–t paths in G', i.e. over all feasible paths. Finally, the path returned via parent pointers is a simple path, so it is a valid answer. ■

Running time

Building G' takes $O(m + n)$ time, since we scan each edge once. Assigning the weights w also takes $O(m)$ time, since each arc is examined once. All weights are nonnegative, so Dijkstra's algorithm applies. With a Fibonacci heap it runs in $O(m + n \log n)$ time (n `Extract-Min` operations at $O(\log n)$ amortized each, and at most $2m$ `Decrease-Key` operations at $O(1)$ amortized each). Recovering the path from the parent pointers takes $O(n)$ time. The total running time is therefore

$$O(m + n) + O(m) + O(m + n \log n) + O(n) = O(m + n \log n)$$